

Terraform-04

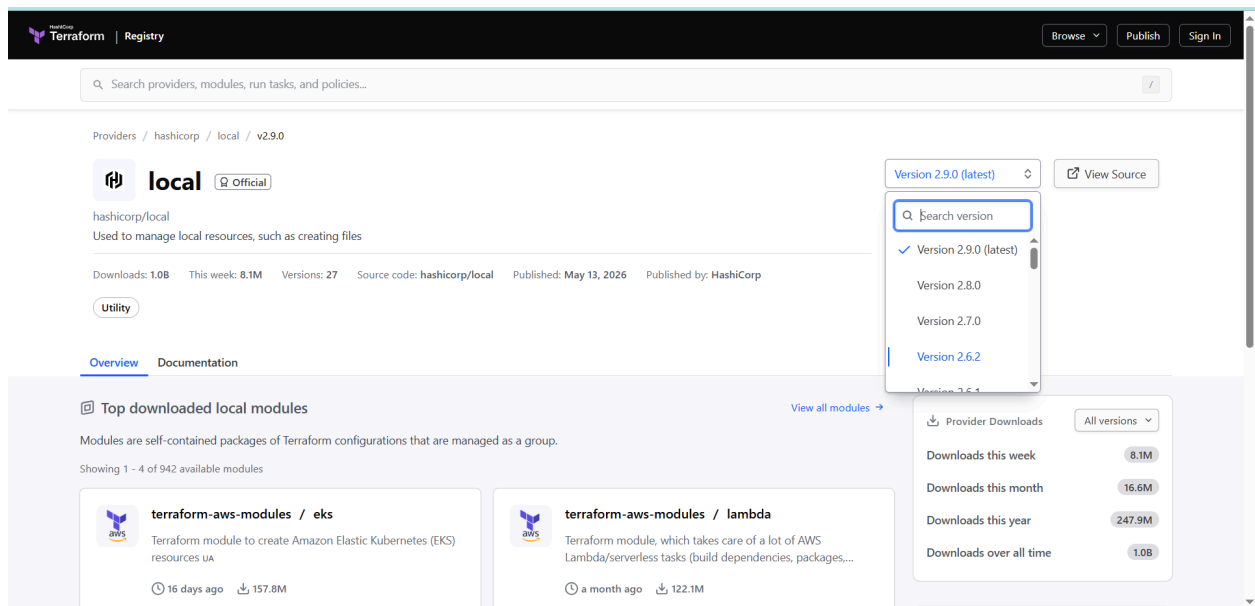
- 1). Watch **Terraform-04** video.
- 2). Execute the script shown in the video.

Version Constraints:

Changing in terraform providers version may get us into incompatibility issues.

By default terraform will always try to download the latest version of provider available from the registry.

To make sure to use the specific version provider we can add the provider block in configuration.



The screenshot shows the Terraform Registry page for the 'local' provider. The page header includes the Terraform logo and 'Registry' link. A search bar is present. The breadcrumb trail is 'Providers / hashicorp / local / v2.9.0'. The provider name 'local' is displayed with an 'Official' badge. Below it, a description states 'Used to manage local resources, such as creating files'. Metadata includes 'Downloads: 1.0B', 'This week: 8.1M', 'Versions: 27', 'Source code: hashicorp/local', 'Published: May 13, 2026', and 'Published by: HashiCorp'. A 'Utility' tag is shown. The 'Overview' tab is selected. A dropdown menu for 'Version 2.9.0 (latest)' is open, showing a search bar and a list of versions: 'Version 2.9.0 (latest)', 'Version 2.8.0', 'Version 2.7.0', and 'Version 2.6.2'. To the right, a 'View Source' button is visible. Below the provider information, the 'Top downloaded local modules' section is shown, with a note that 'Modules are self-contained packages of Terraform configurations that are managed as a group.' and 'Showing 1 - 4 of 942 available modules'. Two modules are listed: 'terraform-aws-modules / eks' (Terraform module to create Amazon Elastic Kubernetes (EKS) resources UA, 16 days ago, 157.8M) and 'terraform-aws-modules / lambda' (Terraform module, which takes care of a lot of AWS Lambda/serverless tasks (build dependencies, packages..., a month ago, 122.1M). On the right side, a 'Provider Downloads' section shows statistics: 'Downloads this week: 8.1M', 'Downloads this month: 16.6M', 'Downloads this year: 247.9M', and 'Downloads over all time: 1.0B'.

16 days ago157.8M

**cloudposse / utils**
Utility functions for use with Terraform in the AWS environment
2 years ago39.8M

a month ago122.1M

**cloudposse / stack-config**
Terraform module that loads an opinionated 'stack' configuration from local or remote YAML sources. It support...
3 months ago29.8M

Helpful Links

[Source Code](#)[Try HCP Terraform](#)[Register for a workshop](#)[Report Issue](#)

[Using providers](#)[View tutorials](#)[Post a forum question](#)

How to use this provider

To install this provider, copy and paste this code into your Terraform configuration. Then, run `terraform init`.

```
Terraform 0.13+

terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "2.9.0"
    }
  }
}

provider "local" {
  # Configuration options
}
```

INTROLEARNDOCSEXTENDCOMMUNITYSTATUSPRIVACYSECURITYTERMPRESS KIT

 © HashiCorp 2026

<https://registry.terraform.io/modules/cloudposse/utils/aws/latest>

FileEditSelectionViewGoRunTerminalHelp

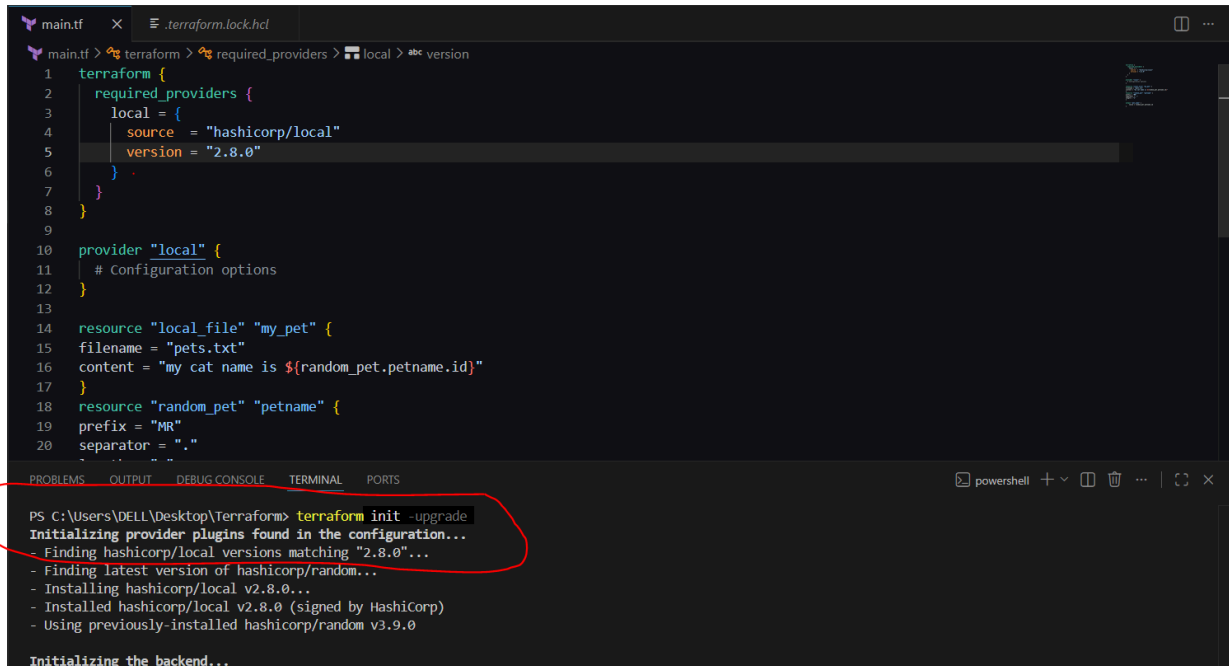
Update

EXPLORERTERRAFORM> .terraform> .terraform.lock.hclmain.tfpets.txtterraform.tfstatevariable.tfterraform.tfstate.backup

main.tf

```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "2.8.0"
6     }
7   }
8 }
9
10 provider "local" {
11   # Configuration options
12 }
13
14 resource "local_file" "my_pet" {
15   filename = "pets.txt"
16   content = "my cat name is ${random_pet.petname.id}"
17 }
18 resource "random_pet" "petname" {
19   prefix = "MR"
20   separator = "."
21   length = "1"
22 }
23
24 output "pet_name" {
25   value = random_pet.petname.id
26 }
```

Upgrading from 2.9 to 2.8:

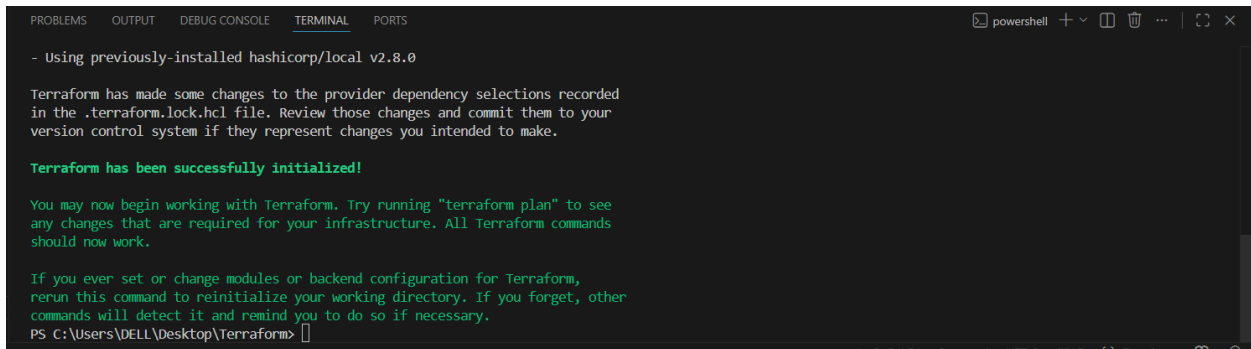


The screenshot shows a VS Code editor with a file named `.terraform.lock.hcl` open. The file contains a Terraform configuration for a local provider. The terminal window at the bottom shows the output of the command `terraform init -upgrade`. A red circle highlights the first line of the terminal output: `Initializing provider plugins found in the configuration...`.

```
main.tf x .terraform.lock.hcl
main.tf > terraform > required_providers > local > abc version
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "2.8.0"
6     }
7   }
8 }
9
10 provider "local" {
11   # Configuration options
12 }
13
14 resource "local_file" "my_pet" {
15   filename = "pets.txt"
16   content = "my cat name is ${random_pet.petname.id}"
17 }
18 resource "random_pet" "petname" {
19   prefix = "MR"
20   separator = "."
21 }
22 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\Desktop\Terraform> terraform init -upgrade
Initializing provider plugins found in the configuration...
- Finding hashicorp/local versions matching "2.8.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.8.0...
- Installed hashicorp/local v2.8.0 (signed by HashiCorp)
- Using previously-installed hashicorp/random v3.9.0

Initializing the backend...
```



This screenshot shows the continuation of the terminal output from the previous screenshot. It displays the message "Terraform has been successfully initialized!" and provides instructions on how to use Terraform.

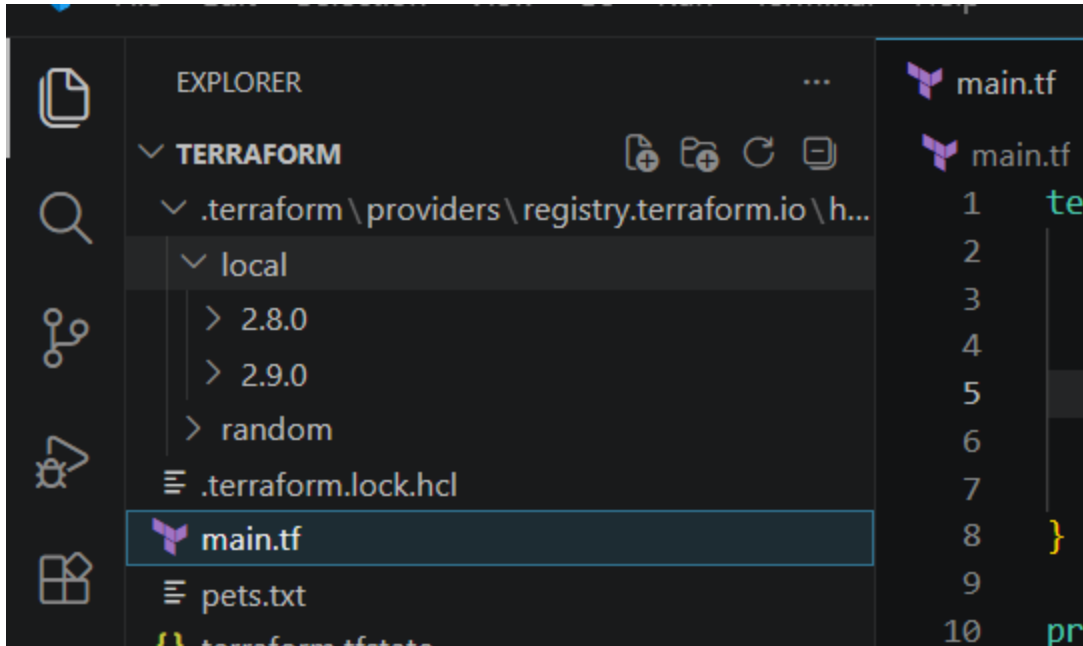
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
- Using previously-installed hashicorp/local v2.8.0

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

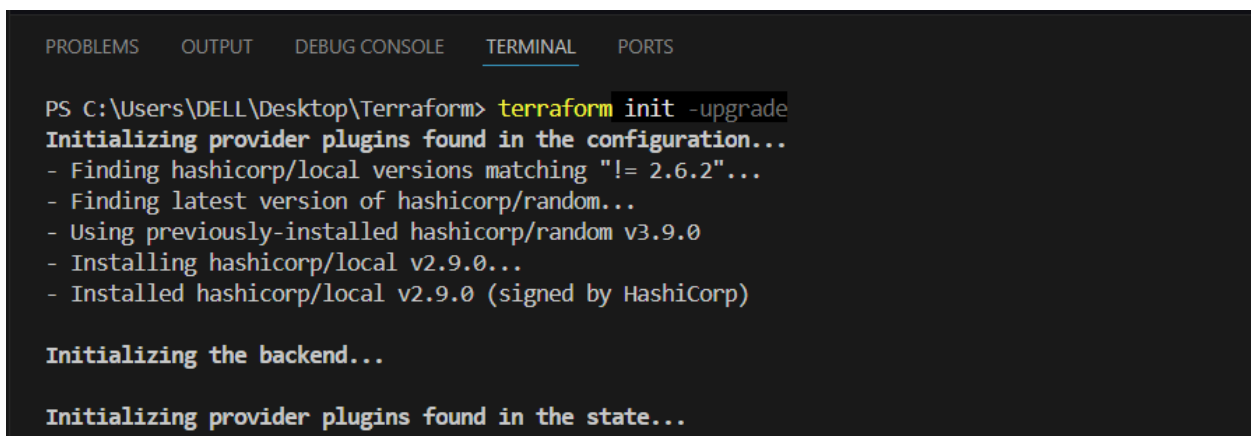
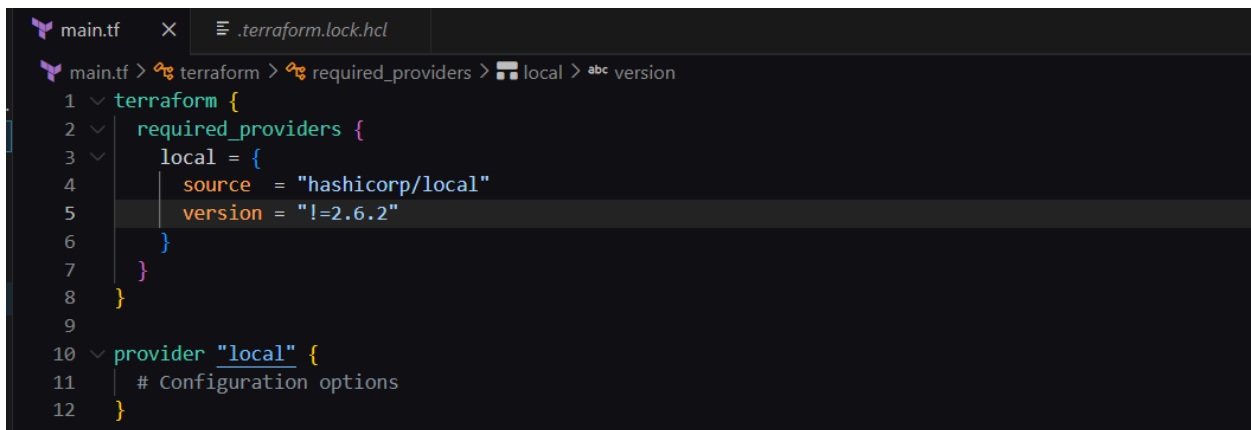
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\DELL\Desktop\Terraform>
```



version = "!=2.6.2"



- Using previously-installed hashicorp/local v2.9.0

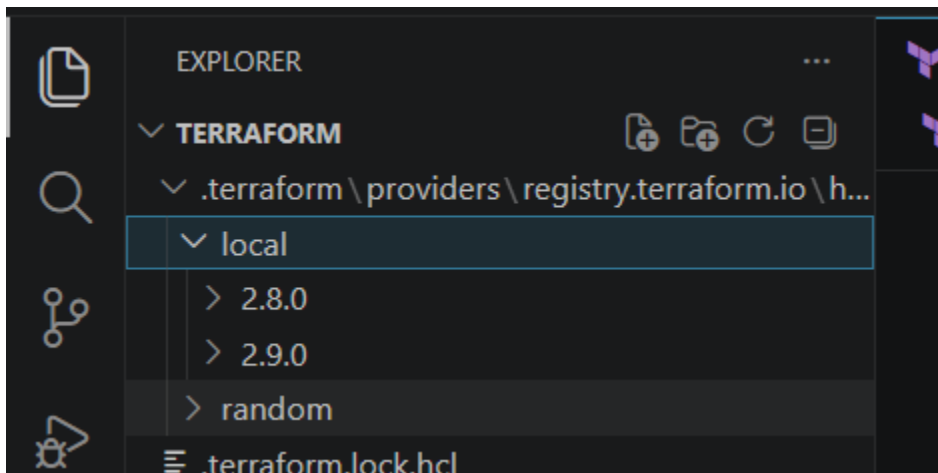
Terraform has made some changes to the provider dependency selections recorded in the .terraform.lock.hcl file. Review those changes and commit them to your version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

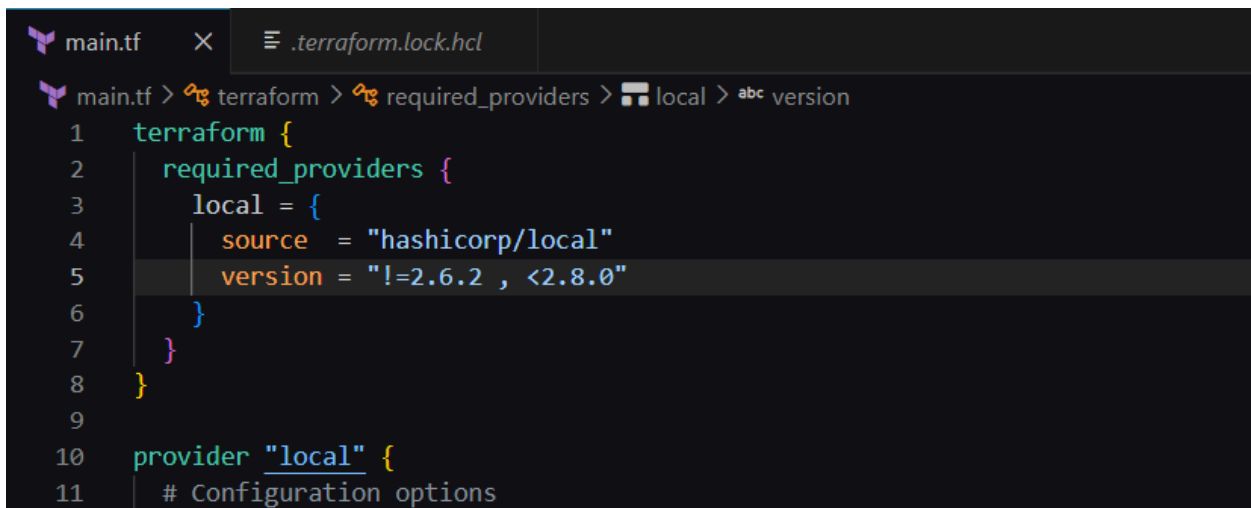
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

PS C:\Users\DELL\Desktop\Terraform>



Passing 2 parameters:



```

PS C:\Users\DELL\Desktop\Terraform> terraform init -upgrade
Initializing provider plugins found in the configuration...
- Finding hashicorp/local versions matching "!= 2.6.2, < 2.8.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.7.0...
- Installed hashicorp/local v2.7.0 (signed by HashiCorp)
- Using previously-installed hashicorp/random v3.9.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/local
- Reusing previous version of hashicorp/random
- Using previously-installed hashicorp/local v2.7.0
- Using previously-installed hashicorp/random v3.9.0

```

The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project structure with a 'TERRAFORM' folder containing a 'providers' subfolder, which in turn contains a 'registry.terraform.io' folder and a 'local' folder. The 'local' folder contains files for versions 2.7.0, 2.8.0, and 2.9.0, as well as a 'random' folder. The main editor pane shows the 'main.tf' file with the following content:

```

1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "!=2.6.2 , <2.8.0"
6     }
7   }
8 }
9
10 provider "local" {

```

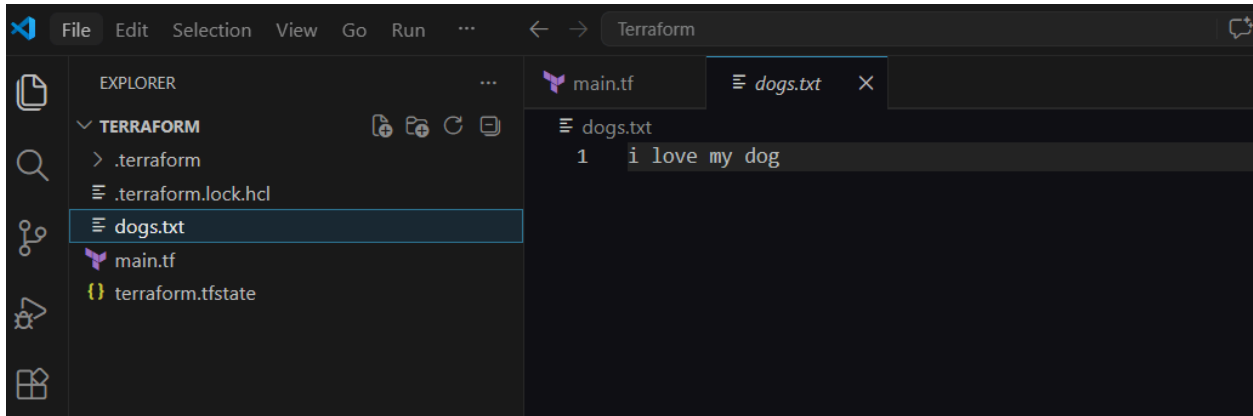
Data variables:

The screenshot shows the Visual Studio Code interface. The main editor pane shows the 'main.tf' file with the following content:

```

1 resource "local_file" "my_pet" {
2   filename = "dogs.txt"
3   content  = data.local_file.cat.content
4 }
5
6 data "local_file" "cat" {
7   filename = "cats.txt"
8 }

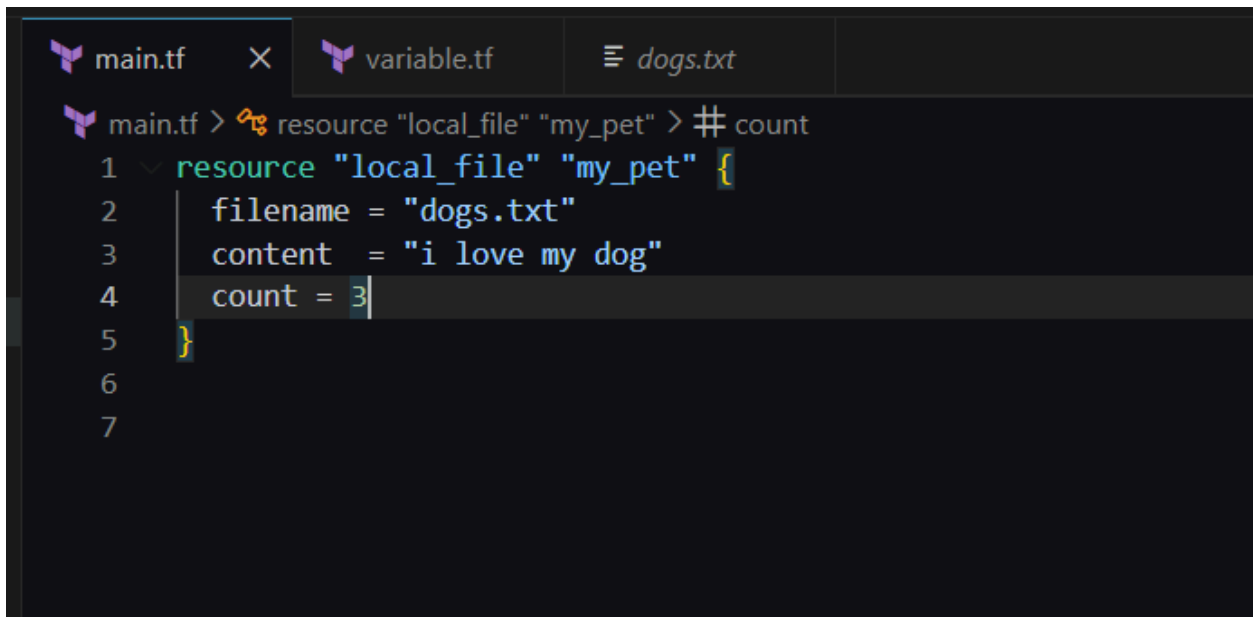
```



Now you can check in pets.txt

It is read from the main.tf and dogs.txt and then output will get in pets.txt

Meta arguments:



It is overriding:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [X] ... | [ ] [X]
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
```

```
Do you want to perform these actions?
```

```
Terraform will perform the actions described above.
```

```
Only 'yes' will be accepted to approve.
```

```
Enter a value: yes
```

```
local_file.my_pet[2]: Creating...
```

```
local_file.my_pet[1]: Creating...
```

```
local_file.my_pet[1]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
```

```
local_file.my_pet[2]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
```

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

```
PS C:\Users\DELL\Desktop\Terraform>
```

```
main.tf > ...
1 resource "local_file" "my_pet" {
2   filename = var.filename[count.index]
3   content  = "i love my dog"
4   count    = 3
5 }
6
7

variable.tf > variable "filename"
1 variable "filename" {
2   default = [
3     "pets.txt",
4     "dogs.txt",
5     "cats.txt"
6   ]
7
8 }
```

```
Enter a value: yes

local_file.my_pet[2]: Destroying... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[0]: Destroying... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[2]: Destruction complete after 0s
local_file.my_pet[0]: Destruction complete after 0s
local_file.my_pet[2]: Creating...
local_file.my_pet[0]: Creating...
local_file.my_pet[2]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[0]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\DELL\Desktop\Terraform> terraform apply
local_file.my_pet[2]: Refreshing state... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[1]: Refreshing state... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[0]: Refreshing state... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
```


Here throughs an error because of in [main.tf](#) file count is 3 and in [variable.tf](#) we have 2 files, thats why throughs error

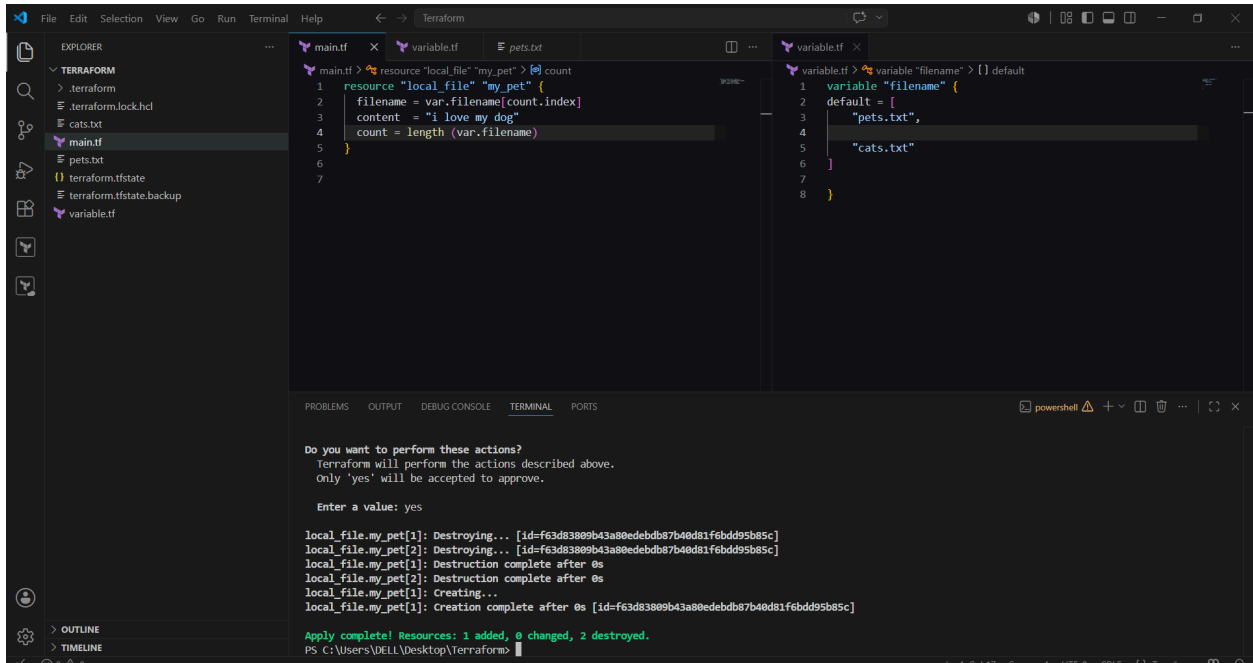
```
File Edit Selection View Go Run ... Terraform
EXPLORER
TERRAFORM
> .terraform
> .terraform.lock.hcl
cats.txt
dogs.txt
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
variable.tf

main.tf > ...
1 resource "local_file" "my_pet" {
2   filename = var.filename[count.index]
3   content  = "i love my dog"
4   count    = 3
5 }
6
7

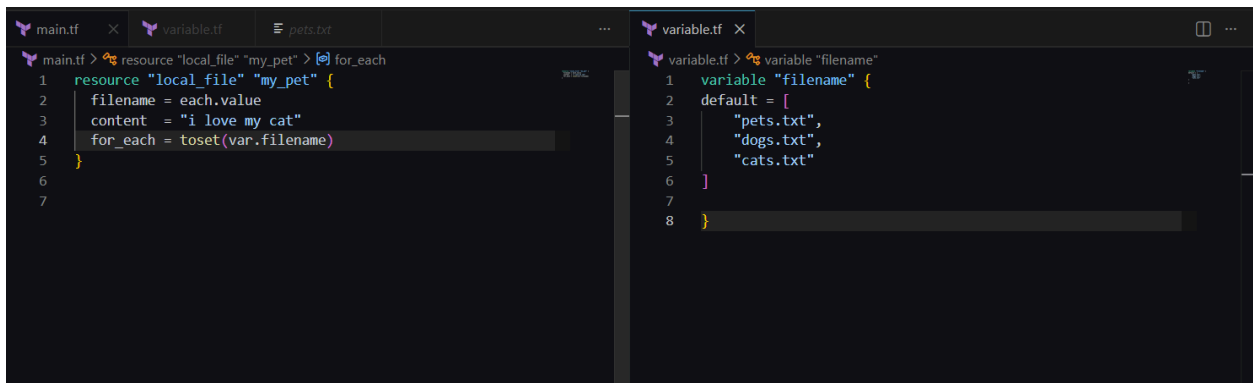
variable.tf > variable "filename" > {} default
1 variable "filename" {
2   default = [
3     "pets.txt",
4
5     "cats.txt"
6   ]
7
8 }

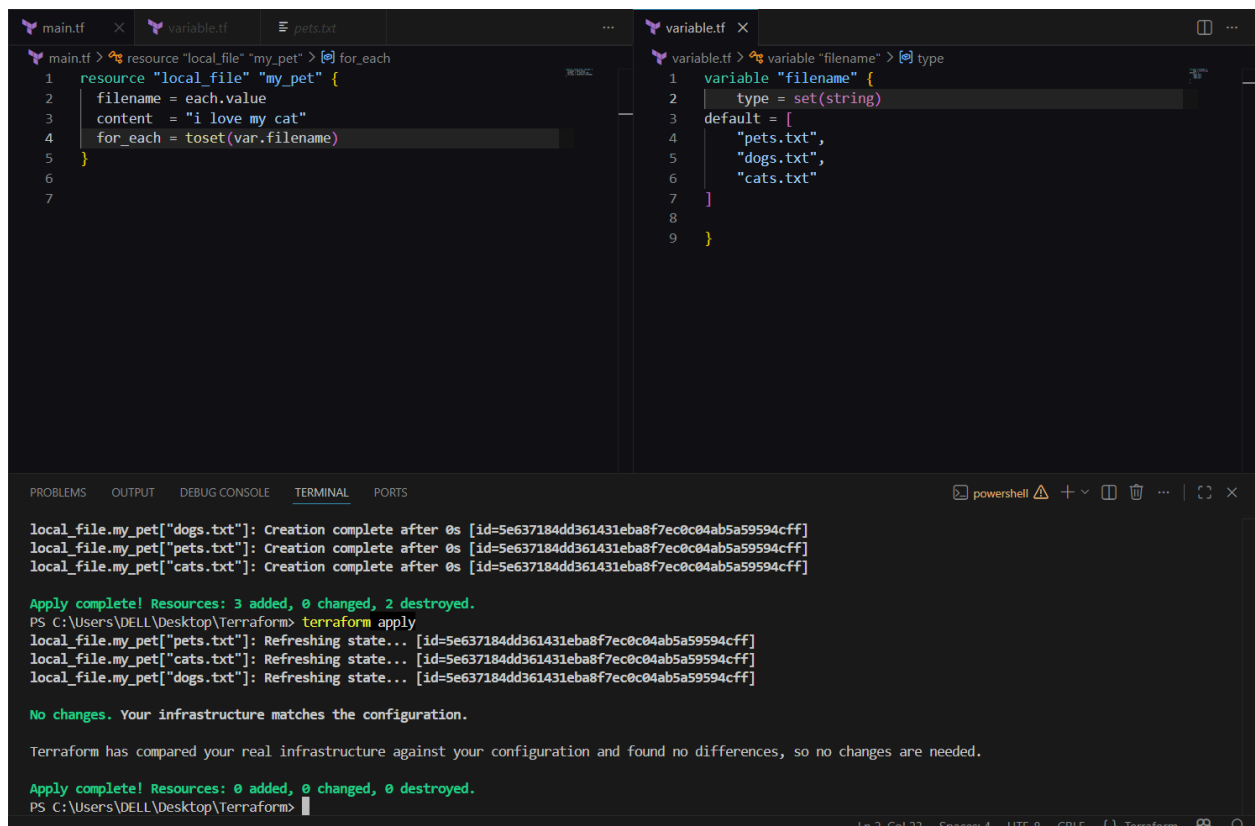
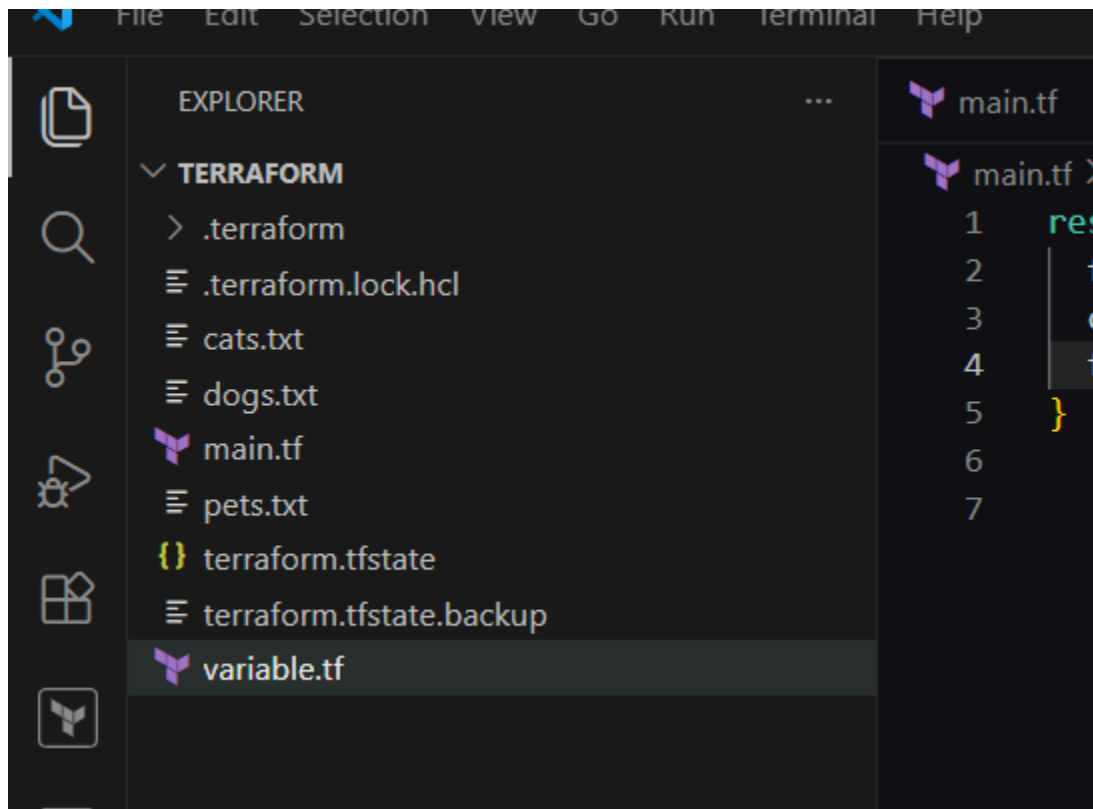
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
}
Plan: 1 to add, 0 to change, 1 to destroy.
Error: Invalid index
on main.tf line 2, in resource "local_file" "my_pet":
2: filename = var.filename[count.index]
   count.index is 2
   var.filename is tuple with 2 elements
The given key does not identify an element in this collection value: the given index is greater than or equal to the length of the collection.
PS C:\Users\DELL\Desktop\Terraform>
```

count = lenght(var.filename):



Using for each:





Creating I am user in EC2 by using Terraform:

Aws configure:

```
PS C:\Users\DELL\Desktop\Terraform> aws configure
AWS Secret Access Key [*****eAv9]: /3zoV08WMQVzG/epY466Iq1PhvvFrTOuj4fDwITe
Default region name [ap-south-1]:
Default output format [json]:
PS C:\Users\DELL\Desktop\Terraform> aws --version
```

Write a script: creating IAM user

```
main.tf  X
main.tf > resource "aws_iam_user" "admin_user" > tags > abc description
1  resource "aws_iam_user" "admin_user" {
2    name = "bhargav"
3    tags = {
4      description = "technical team lead"
5    }
6  }
```

Terraform init:

```

PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

```

Terraform plan:

```

Commands will detect it and remind you to do so if necessary.
PS C:\Users\DELL\Desktop\Terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_iam_user.admin_user will be created
+ resource "aws_iam_user" "admin_user" {
  + arn          = (known after apply)
  + force_destroy = false
  + id           = (known after apply)
  + name        = "bhargav"
  + tags        = {
    + "description" = "technical team lead"
  }
}

```

Terraform apply:

```

PS C:\Users\DELL\Desktop\Terraform> terraform apply

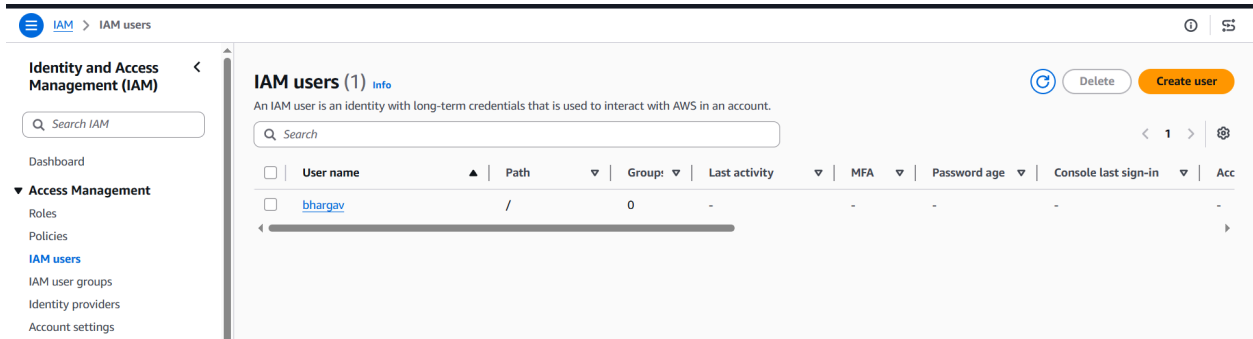
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_iam_user.admin_user will be created
+ resource "aws_iam_user" "admin_user" {
  + arn          = (known after apply)
  + force_destroy = false
  + id           = (known after apply)
  + name        = "bhargav"
  + path        = "/"
  + tags        = {
    + "description" = "technical team lead"
  }
}

```

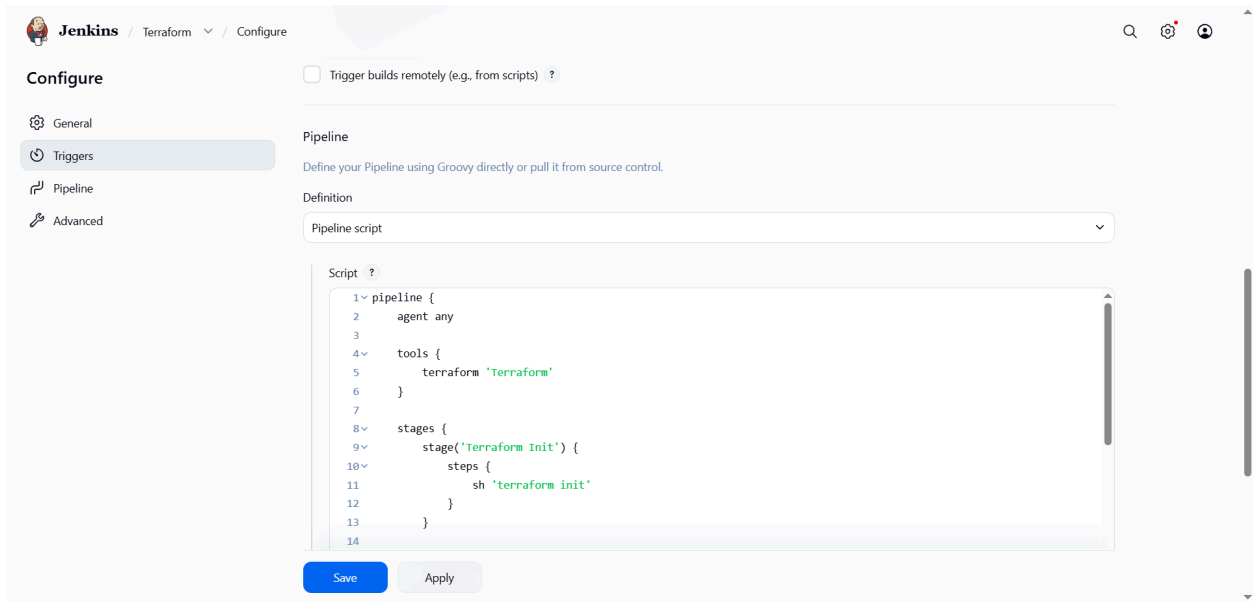
Verify : automatically create a new user



3). Integrate Terraform in Jenkins using the Terraform plugin.

```
[root@ip-172-31-14-229 ~]# cd /var/lib/jenkins/workspace/Terraform
[root@ip-172-31-14-229 Terraform]# ls
main.tf  terraform-from-jenkins.txt  terraform.tfstate
[root@ip-172-31-14-229 Terraform]# cat main.tf
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content  = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
```



```
pipeline {
  agent any

  tools {
    terraform 'Terraform'
  }

  stages {
    stage('Terraform Init') {
      steps {
        sh 'terraform init'
      }
    }

    stage('Terraform Plan') {
      steps {
        sh 'terraform plan'
      }
    }

    stage('Terraform Apply') {
      steps {
        sh 'terraform apply -auto-approve'
      }
    }
  }
}
```



The screenshot shows the Jenkins web interface with a console output for a Terraform apply command. The output includes a plan for creating a local file and the successful execution of the apply command.

```
jenkins / Terraform / #5

[32m+[0m[0m content_sha256      = (known after apply)
[32m+[0m[0m content_sha512        = (known after apply)
[32m+[0m[0m directory_permission = "0777"
[32m+[0m[0m file_permission       = "0777"
[32m+[0m[0m filename               = "terraform-from-jenkins.txt"
[32m+[0m[0m id                    = (known after apply)
}

[1mPlan:[0m [0m [0m1 to add, 0 to change, 0 to destroy.
[0m[0m[1mlocal_file.demo: Creating...[0m[0m
[0m[0m[1mlocal_file.demo: Creation complete after 0s [id=221065dac85086259e58924cd5d80a552d879e04][0m
[0m[0m[1m[0m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

4). Create a **CI/CD pipeline** for a Nodejs Application:

<https://github.com/betawins/Trading-UI.git>

Status of jenkins

```
[root@ip-172-31-14-229 ~]# systemctl status jenkins
• jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-06-11 10:21:59 UTC; 4 days ago
     Main PID: 3032 (java)
        Tasks: 46 (limit: 4543)
       Memory: 1003.0M
          CPU: 21min 45.879s
      CGroup: /system.slice/jenkins.service
              └─3032 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --we

Jun 14 10:22:13 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-14 10:22:13.
Jun 14 10:22:13 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-14 10:22:13.
Jun 14 17:37:53 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-14 17:37:53.
Jun 15 10:22:01 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:01.
Jun 15 10:22:02 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:02.
Jun 15 10:22:04 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:04.
Jun 15 10:22:06 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:06.
Jun 15 10:22:07 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:07.
```

Install nodejs: `dnf install nodejs -y`


```
[root@ip-172-31-14-229 ~]# dnf install nodejs -y
Last metadata expiration check: 8:20:24 ago on Mon Jun 15 02:04:42 2026.
Dependencies resolved.
=====
Package                                Architecture      Version
=====
Installing:
nodejs                                x86_64            1:18.20.8-1.amzn2023.0.2
Installing dependencies:
nodejs-libs                           x86_64            1:18.20.8-1.amzn2023.0.2
Installing weak dependencies:
nodejs-docs                           noarch            1:18.20.8-1.amzn2023.0.2
nodejs-full-i18n                       x86_64            1:18.20.8-1.amzn2023.0.2
nodejs-npm                             x86_64            1:10.8.2-1.18.20.8.1.amzn2023.0.2
=====
Transaction Summary
=====
Install 5 Packages

Total download size: 45 M
Installed size: 223 M
Downloading Packages:
(1/5): nodejs-docs-18.20.8-1.amzn2023.0.2.noarch.rpm
(2/5): nodejs-18.20.8-1.amzn2023.0.2.x86_64.rpm
(3/5): nodejs-full-i18n-18.20.8-1.amzn2023.0.2.x86_64.rpm
```

verify:

```
[root@ip-172-31-14-229 ~]# node -v
v18.20.8
```

Install pm2: npm install -g pm2

```
[root@ip-172-31-14-229 ~]# npm install -g pm2

added 90 packages in 4s

8 packages are looking for funding
  run `npm fund` for details

npm notice
npm notice New major version of npm available! 10.8.2 -> 11.17.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.17.0
npm notice To update run: npm install -g npm@11.17.0
npm notice
```

verify:

[illegible]

Create new item:

 **Jenkins** / All / New Item

New Item

Enter an item name

Select an item type



Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

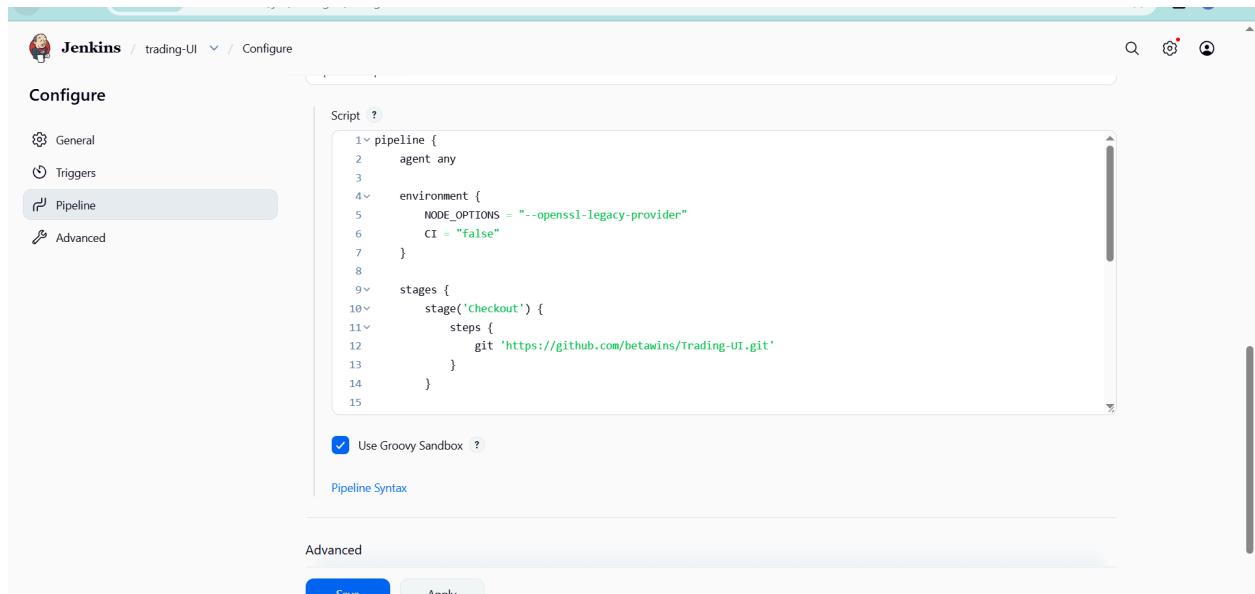


Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.



Multi-configuration project

Write a pipeline:



```
pipeline {
  agent any
```

```
  environment {
    NODE_OPTIONS = "--openssl-legacy-provider"
    CI = "false"
  }
```

```
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/betawins/Trading-UI.git'
      }
    }
  }
```

```
    stage('Install & Build') {
      steps {
        sh '''
        npm install
        npm run build
        '''
```

```

    }
}

stage('Deploy') {
    steps {
        sh '''
            pm2 restart Trading-UI || pm2 start npm --name Trading-UI --
start
            '''
        }
    }
}
}

```

Save and build

Build successful


Jenkins / trading-UI / #1

Status

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

✓

#1 (15 Jun 2026, 10:38:01)

Started by user [bhargav radharapu](#)

This run spent:

- 13 ms waiting;
- 2 min 16 sec build duration;
- 2 min 16 sec total from scheduled to completion.

git

Revision: d3555e39b908103ac6d099560b70a64e6dee18f0

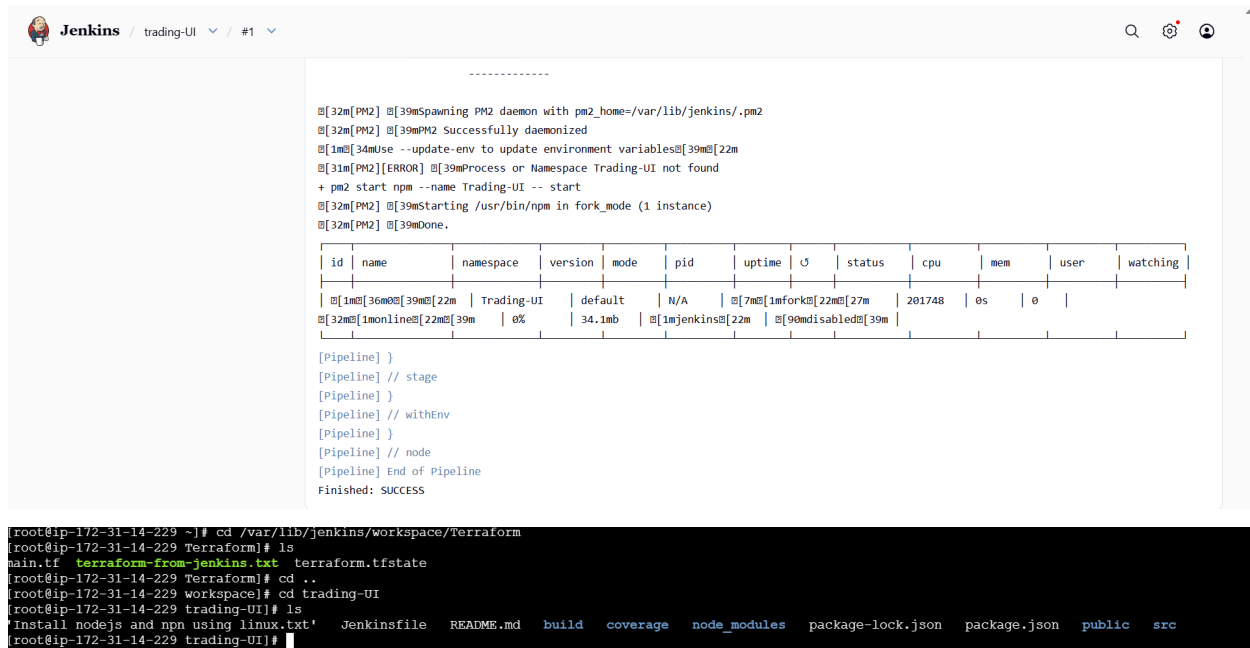
Repository: <https://github.com/betawins/Trading-UI.git>

- refs/remotes/origin/master

</>

No changes.

Verify output:



```
-----
[32m[PM2] [39mSpawning PM2 daemon with pm2_home=/var/lib/jenkins/.pm2
[32m[PM2] [39mPM2 Successfully daemonized
[1m[34mUse --update-env to update environment variables[39m[22m
[31m[PM2][ERROR] [39mProcess or Namespace Trading-UI not found
+ pm2 start npm --name Trading-UI -- start
[32m[PM2] [39mStarting /usr/bin/npm in fork_mode (1 instance)
[32m[PM2] [39mDone.

| id | name | namespace | version | mode | pid | uptime |  | status | cpu | mem | user | watching |
|----|-----|-----|-----|-----|-----|-----|---|-----|----|----|-----|-----|
| [1m[36m[39m[22m | Trading-UI | default | N/A | [7m[1mfork[22m[27m | 201748 | 0s | 0 |  |  |  |  |  |
| [32m[1monline[22m[39m | 0% | 34.1mb | [1mjenkins[22m | [90mdisabled[39m |  |  |  |  |  |  |  |

[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

[root@ip-172-31-14-229 ~]# cd /var/lib/jenkins/workspace/Terraform
[root@ip-172-31-14-229 Terraform]# ls
main.tf terraform-from-jenkins.txt terraform.tfstate
[root@ip-172-31-14-229 Terraform]# cd ..
[root@ip-172-31-14-229 workspace]# cd trading-UI
[root@ip-172-31-14-229 trading-UI]# ls
'Install nodejs and npm using linux.txt' Jenkinsfile README.md build coverage node_modules package-lock.json package.json public src
[root@ip-172-31-14-229 trading-UI]#
```

5). Explain 10 Maven commands

Apache Maven is a popular Java build and dependency management tool. Its

commands are run through the mvn CLI. Here are 10 important Maven commands and what they do:

1. mvn clean

Removes the target/ directory where compiled files and build outputs are stored. Useful to ensure a fresh build without old artifacts interfering.

2. mvn compile

Compiles the source code of your project (src/main/java). It does not run tests

or package the application—only checks and compiles code.

3. mvn test

Runs unit tests defined in src/test/java using testing frameworks like JUnit.

It

compiles both main code and test code before execution.

4. mvn package

Takes compiled code and packages it into a distributable format like a JAR or

WAR file inside the target/ directory.

5. mvn install

Installs the packaged artifact into your local Maven repository (~/.m2/repository) so it can be used as a dependency in other local projects.

6. mvn deploy

Copies the final package to a remote repository (like Nexus or Artifactory), making it available for other developers or CI/CD pipelines.

7. mvn validate

Checks whether the project structure and configuration are correct before the

build starts. It ensures the project is ready to build.

8. mvn verify

Runs additional checks on the package after tests, such as integration tests or

quality checks, to ensure the build is valid.

9. mvn dependency:tree

Displays the full dependency hierarchy of the project. Helpful for debugging version conflicts or unwanted transitive dependencies.

10. mvn archetype:generate

Creates a new Maven project from a template (archetype). It helps quickly scaffold standard project structures